

CWE — Study Guide

CWE (Common Weakness Enumeration) is the classification system for **types of vulnerabilities** — where CVE identifies one *specific instance* of a flaw in one *specific product*, CWE identifies the *underlying category/root cause* that instance belongs to. This is the piece that turns a list of individual CVEs into actual pattern recognition — genuinely valuable for both pentesting (recognizing vulnerability classes) and secure development.

1. CVE vs. CWE — The Core Distinction (Get This Cold)

```
CVE-2021-44228 (Log4Shell) → ONE specific vulnerability, in ONE specific product (Apache Log4j)
CWE-917 → the GENERAL CATEGORY it belongs to:
"Improper Neutralization of Special Elements used in an Expression Language Statement" (a form of injection)
```

Think of it like this:

```
CVE = a specific crime that happened, with a case number
CWE = the criminal charge/category that crime falls under
(burglary, fraud, etc.)
```

Every CVE typically gets mapped to one or more CWE categories — this is exactly the **CWE:** field you'd see in an NVD record alongside the CVSS score.

2. Why CWE Matters More Than It First Seems

Learning individual CVEs teaches you about specific historical bugs. Learning CWE categories teaches you to **recognize the underlying pattern** wherever it shows up next — a skill that transfers to code you've never seen and vulnerabilities that haven't been discovered yet. This is precisely the mindset shift from "I know EternalBlue" to "I understand memory corruption vulnerabilities and can reason about new ones."

3. CWE ID Format

CWE-NNN

CWE-79 → Cross-Site Scripting (XSS)
CWE-89 → SQL Injection
CWE-119 → Improper Restriction of Operations within Memory Buffer Bounds
CWE-798 → Use of Hard-coded Credentials

Unlike CVE, CWE numbers aren't tied to a year — they're a fixed, hierarchical taxonomy maintained by MITRE, updated/expanded periodically but not tied to disclosure timing.

4. CWE Structure — It's a Hierarchy, Not a Flat List

CWE entries are organized in **parent-child relationships**, from broad categories down to very specific weakness types:

```
CWE-707 (Improper Neutralization) [broad category]
├─ CWE-74 (Injection) [more specific]
│   ├─ CWE-89 (SQL Injection) [very specific]
│   ├─ CWE-79 (Cross-Site Scripting) [very specific]
│   └─ CWE-78 (OS Command Injection) [very specific]
```

Understanding this hierarchy helps you see that XSS, SQLi, and command injection are all fundamentally the **same underlying problem** (untrusted input reaching a context where it's interpreted as code/commands instead of data) — just manifesting in different contexts.

5. The CWE Top 25 Most Dangerous Software Weaknesses

MITRE publishes an annual **CWE Top 25** — ranked by real-world prevalence and severity, calculated from actual CVE data. This is genuinely worth memorizing the top handful of, since these show up constantly:

Rank (typical)	CWE	Weakness
1	CWE-79	Cross-Site Scripting (XSS)

Rank (typical)	CWE	Weakness
2	CWE-787	Out-of-bounds Write
3	CWE-89	SQL Injection
4	CWE-352	Cross-Site Request Forgery (CSRF)
5	CWE-22	Path Traversal
6	CWE-125	Out-of-bounds Read
7	CWE-78	OS Command Injection
8	CWE-416	Use After Free
9	CWE-862	Missing Authorization
10	CWE-434	Unrestricted Upload of File with Dangerous Type

(Exact rankings shift slightly year to year — worth checking cwe.mitre.org for the current published list rather than treating this as fixed.)

6. CWEs You've Already Encountered Throughout This Series (Now Named Explicitly)

This is where CWE really clicks — connecting it back to everything you've already studied:

Concept From Earlier Guides	CWE
SQL error messages found via Google Dorking	CWE-89 (SQL Injection)
<code>.env</code> /credential files exposed via dorking/GitHub	CWE-798 (Use of Hard-coded Credentials), CWE-200 (Exposure of Sensitive Information)
Directory listing (<code>intitle:"index of"</code>)	CWE-548 (Exposure of Information Through Directory Listing)
Unquoted service paths (Windows privesc)	CWE-428 (Unquoted Search Path or Element)
SUID binary privilege escalation	CWE-269 (Improper Privilege Management)
Writable cron scripts	CWE-732 (Incorrect Permission Assignment for Critical Resource)
IDOR (from your CIA Triad guide)	CWE-639 (Authorization Bypass Through User-Controlled Key)

Concept From Earlier Guides	CWE
Missing security headers (Nikto findings)	CWE-693 (Protection Mechanism Failure)
Anonymous LDAP/null SMB sessions	CWE-306 (Missing Authentication for Critical Function)
Weak/no lockout policy (Enum4linux finding)	CWE-307 (Improper Restriction of Excessive Authentication Attempts)
EternalBlue's underlying flaw	CWE-119 (Improper Restriction of Operations within Memory Buffer Bounds)

Seeing this table should genuinely reframe a lot of what you've studied — nearly every "finding" across your whole series maps to a small, recurring set of weakness categories, which is the entire point of CWE existing.

7. Looking Up CWE Details

bash

```
https://cwe.mitre.org/data/definitions/89.html      # CWE-
89, SQL Injection, full detail page
```

Each CWE page includes: description, common consequences, applicable platforms, example vulnerable code, detection methods, and related CWEs (parent/child relationships) — genuinely worth reading a few of these in full rather than just knowing the ID/name, since the "example vulnerable code" sections are excellent for building actual pattern recognition.

8. CWE in Practice — Where You'll Use It

a) Reading NVD/CVE records — every enriched CVE entry lists its CWE mapping(s):

json

```
"weaknesses": [
  {
    "description": [
      {"lang": "en", "value": "CWE-119"}
    ]
  }
]
```

```
}  
]
```

b) Writing VAPT reports — professional reports increasingly tag findings with both CVE (if applicable) and CWE, since CWE applies even to custom application vulnerabilities that will never get their own CVE (a business logic flaw in a client's internal app won't have a CVE, but it absolutely has a CWE classification).

c) SAST/DAST tool output — static/dynamic code analysis tools (Semgrep, SonarQube, Checkmarx) classify their findings by CWE, since they're finding *patterns in code*, not specific named CVEs.

d) OWASP Top 10 mapping — the OWASP Top 10 web vulnerability categories are essentially a curated, web-focused subset/grouping of relevant CWEs:

OWASP A03:2021 (Injection) ↔ CWE-89, CWE-79, CWE-78, and others

OWASP A01:2021 (Broken Access Control) ↔ CWE-639, CWE-862, CWE-284

9. CWE vs. CVE vs. CAPEC — The Full MITRE Family (Worth Knowing Exists)

MITRE maintains a related trio of frameworks that connect neatly together:

CWE → the type of **weakness** (root cause category)

CVE → a specific instance of that weakness, in specific software

CAPEC → Common Attack Pattern Enumeration and Classification — describes **HOW** an attacker actually exploits a given **CWE**, from the attacker's perspective

CWE-89 (SQL Injection)

→ **CVE-2021-XXXXX** (a specific SQLi vulnerability in Product X)

→ **CAPEC-66** (**SQL** Injection attack pattern – describes the actual attacker technique/methodology)

CAPEC is less commonly referenced day-to-day than CVE/CWE, but worth knowing it exists as "the attacker's playbook" layer connecting weakness types to actual exploitation methodology — conceptually similar to how ATT&CK describes technique execution.

10. Practical Exercise — Mapping Your Own Findings

A genuinely valuable habit going forward: whenever you find something in a lab/CTF/VulnHub box, tag it with a CWE (in addition to the ATT&CK technique ID habit from your earlier guides):

Finding: **SQL** injection in the login form of **DVWA**
CWE: **CWE-89**
ATT&CK: **T1190** (Exploit **Public**-Facing Application)
CVSS: (calculate based on the specific instance's impact)

This three-part tagging — CWE (weakness type) + ATT&CK (adversary technique) + CVSS (severity) — is close to how mature security teams actually classify findings, and demonstrating this habit in your VulnHub-Walkthroughs writeups would set them apart from a typical "here's how I got root" narrative.

11. Kill Chain / ATT&CK Mapping

CWE isn't itself a Kill Chain stage or ATT&CK technique — it's an **orthogonal classification layer** describing *why* a vulnerability exists (root cause), which sits alongside CVE (*what* specific instance) and ATT&CK (*how* an adversary would exploit it operationally). All three together give a complete picture of any given finding.

Quick Reference Cheat Sheet

Format: **CWE-NNN** (no year, fixed hierarchical taxonomy)
Structure: broad **parent** categories → specific child weaknesses
Top **25**: published annually by **MITRE**, ranked by real-world prevalence/severity

Common ones to know cold:

CWE-79	XSS
CWE-89	SQL Injection
CWE-78	OS Command Injection
CWE-22	Path Traversal
CWE-352	CSRF
CWE-798	Hard-coded Credentials
CWE-306	Missing Authentication
CWE-862	Missing Authorization

CVE = specific instance, in specific software, dated
CWE = general weakness category, undated, hierarchical
CAPEC = the attacker's methodology **for** exploiting a given
CWE